

Java Interview Q&A

Interviewer: Basically, what is the difference between JDK, JRE, and JVM?

Candidate: So, basically, JDK means Java Development Kit, JRE means Java Runtime Environment, and JVM means Java Virtual Machine. JDK contains our compiling and execution part; it is responsible for the developer side because a developer has to compile the code, rectify the errors, and run the program. Now, JRE contains the execution part for the customer side. Suppose we give an application to a customer to run; we give them the JRE. So, JDK is for the developer, but for the customer side, JRE is enough. When we install JDK, JRE will also be installed.

Now, talking about JVM: it is the Java Virtual Machine. A virtual machine is created within the operating system. If you install Java on Windows, it creates a virtual machine for Windows. If you install it on Linux, it creates one for Linux. JVM is responsible to execute and run the Java code. When we compile the code, a .class file is created. With the help of the ClassLoader, the JVM accepts this bytecode and executes it on the specific system.

Interviewer: What is public static void main?

Candidate: Actually, public means direct access to the JVM. static means the JVM can call the method without the creation of an object. void means the function will not return anything. main is the calling part or the environment for the Java program; by using this function, we can call different methods. String[] args is used because it is a string array, and any number and any type of arguments we can set.

Interviewer: Can we execute a program without a main method?

Candidate: Actually, in JDK 1.6, it was possible to execute a program without a main method. However, from JDK 1.7 onwards, it is compulsory to write the main method to execute the program.

Interviewer: Is Java a platform-independent language?

Candidate: Yes, Java is a platform-independent language because we compile Java code into bytecode, and .class files are created. This bytecode is responsible for running on every JVM. If we have different systems having JVMs, we can copy the .class file from one system to another system and it will run. So, this bytecode is responsible for the "write once, run anywhere" philosophy.

Interviewer: Is Java a "pure" object-oriented language?

Candidate: Actually, Java is not considered a pure object-oriented language because several features like multiple inheritance are not supported in Java. But if we are

comparing it with C++, we can say it is more object-oriented because it doesn't use things like header files or global variables in the same way.

Interviewer: What is the difference between Stack and Heap memory?

Candidate: Basically, Heap memory is used when an object is created. The object, its instance variables, and the values are stored in the Heap memory. Stack memory is generally for the local area. Local variables declared inside methods are stored there. Suppose when a method is called, its records and variables are pushed into the Stack, and once the work is done, they are removed from the Stack memory.

Interviewer: What is the difference between the equals() method and the == operator?

Candidate: In Java, the equals() method is meant for content comparison, while the == operator is meant for address comparison. For example, == compares if they point to the same memory address, while equals() checks the actual content stored in the object.

Interviewer: Why are pointers not used in Java?

Candidate: Actually, pointers are internally implemented in Java, but they are not provided for the user. This is to keep Java simple and secure. To avoid the complexities found in C++, features like multiple inheritance and pointers were restricted to make the language easier for the user.

Interviewer: What is the use of the JIT compiler?

Candidate: Actually, JIT stands for the Just-In-Time compiler. It is a part of the Java environment that improves performance. It compiles the bytecode into native machine code at runtime, which makes the execution faster for the end user.

Interviewer: What is the difference between local and static variables?

Candidate: Basically, instance variables are created when an object is created. Local variables are variables declared inside a method and are limited to that method's scope; they do not have default values. Static variables (class variables) are stored in the class area and are shared among all instances. They are initialized only once when the class is loaded.

Interviewer: What is the difference between assignment and initialization?

Candidate: Assignment means we first create the variable and then later we assign a value to it. Initialization means at the time of creating the variable, we assign the value immediately.

Interviewer: How do we declare a constant in Java?

Candidate: In Java, we use the final keyword to declare constants. In C++, there is a const keyword, but in Java, const is not implemented for this. We use final. For example, final int A = 10.

Interviewer: Can we declare classes and methods as final?

Candidate: Yes. If a class is final, it means we cannot inherit it; inheritance is restricted. If a method is final, it cannot be overridden. If a variable is final, its value cannot be changed.

Interviewer: What is the difference between break and continue statements?

Candidate: Both are part of the transfer statements. Suppose I want to print numbers 1 to 10. If I use break when the value is 5, the loop stops entirely and the output is 1, 2, 3, 4. If I use continue when the value is 5, it skips only that particular iteration. The output would be 1, 2, 3, 4, 6, 7, 8, 9, 10.

Interviewer: What are the types of loops in Java?

Candidate: Java has four types of loops: for, while, do-while, and the for-each loop. The for-each loop is generally used with arrays or collections to iterate over variables.

Interviewer: Define ClassLoaders.

Candidate: The ClassLoader is a part of the Java Runtime Environment. There are three types: Bootstrap ClassLoader, Extension ClassLoader, and System (or Application) ClassLoader. When we compile Java code, it generates a .class file. The ClassLoader is responsible for loading these .class files into the JVM so the code can be executed.

Interviewer: What is Method Overloading and Method Overriding?

Candidate: These are parts of polymorphism. Method Overloading is an example of compile-time polymorphism; it happens within the same class where methods have the same name but different arguments. Method Overriding is runtime polymorphism; it requires an inheritance relationship between a parent and child class where the method has the same name and the same parameters.

Interviewer: What are the differences between C++ and Java?

Candidate: Some major differences are that Java does not support multiple inheritance or pointers, while C++ does. Java does not use the goto keyword. Java has a built-in Garbage Collector for memory management. Also, Java uses packages to organize code, while C++ uses header files.

Interviewer: What is the difference between final, finally, and finalize?

Candidate: These names are similar but their uses are different. final is a keyword used for restrictions on classes, methods, and variables. finally is a block used in exception

handling that will execute regardless of whether an exception is handled or not; it is used for cleanup code like closing database connections. `finalize` is a method called by the Garbage Collector just before an object is destroyed to perform cleanup activities.

Interviewer: Explain type casting in Java.

Candidate: Type casting means when we want to convert one data type into another. There are two types: Implicit and Explicit. Implicit casting (widening) is done by the compiler when converting a lower data type to a higher one. Explicit casting (narrowing) is a forceful conversion done by the developer when converting a higher data type to a lower one, like converting a `double` to an `int`.

Interviewer: What are the 8 primitive data types and their default values?

Candidate: The 8 types are `byte`, `short`, `int`, `long`, `float`, `double`, `char`, and `boolean`. In Java, whole numbers are treated as `int` by default and decimal numbers as `double`. A very important point here: in Java, a `char` takes 2 bytes of memory because it uses the Unicode system. C++ uses the ASCII system where a `char` is only 1 byte. The Unicode system is used in Java to support international languages and a wider range of characters.

Interviewer: Define Exception Handling in Java.

Candidate: Exception handling is used to maintain the normal flow of the program even if runtime errors occur. There are Checked exceptions, which are checked by the compiler at compile-time, and Unchecked exceptions, which occur at runtime. We use five keywords: `try`, `catch`, `finally`, `throw`, and `throws`. Some examples of exceptions are `ArithmeticException` (like dividing by zero), `ArrayIndexOutOfBoundsException`, `FileNotFoundException`, and `ClassNotFoundException`.

Interviewer: Can we overload the main method?

Candidate: Yes, we can overload the main method. We can have multiple main methods with different arguments.

Interviewer: Can we override a static method?

Candidate: No, we cannot override static methods. This is because static methods are part of the class, not the object, and they are not part of runtime polymorphism.

Interviewer: What are the four types of access modifiers?

Candidate:

1. **Public:** Accessible everywhere outside the class.
2. **Private:** Accessible only within that specific class.

3. **Protected:** Accessible within the same package and also in the child class through inheritance.
4. **Default (Friendly):** If we are not mentioning any keyword like public or private, then by default, it is the "Default" or "Friendly" modifier. It is accessible only within the same package.

Interviewer: Is size necessary when declaring an array?

Candidate: Yes, size is necessary when you are allocating the memory. When we create an array, we have to pass the size so the JVM knows how much memory to allocate.

Java OOPs

Interviewer: What is a class and object? **Candidate:** A class is a blueprint or template for object and we can say that it is a logical representation of things. And what's an object is a physical representation of class or physical imagination of things. Means if you want to access your data member of class we have to create the object. Means class model has been created. For example, we have a classroom and we have a college. So college having number of classrooms and students who have not taken admission in schools or college, so this is class data and logically. Technical definition of object is: object is the instance of a class.

Interviewer: Tell me about the Object class in Java. **Candidate:** Object class is a super class of every in Java, all classes and predefined classes. Definition of class is this method a very much important role class, like for example finalize method, toString method, notify, notifyAll method, wait method and equals method. So these all are compulsory and important for every class.

Interviewer: Is Java a pure object-oriented language? **Candidate:** Java is a object oriented language because there is special features of object oriented programming. If a language supports features to be considered this particular language is the different different language. For example, we have number of features which support the real world objects and life, for example we have class, object, inheritance, polymorphism, message passing, dynamic binding, abstraction, and encapsulation. So these all features are supported by programming languages. Java and C++ is object oriented language and C is procedural language.

Interviewer: What is the difference between procedural and object-oriented programming? **Candidate:** In procedural programming, logic follows setting procedures and functions and in object oriented programming we follow the real world objects entity like for example class, objects, inheritance, and polymorphism. Procedural programming follows top-to-bottom approach and object oriented programming follows bottom-to-top approach. In procedural we don't have access modifiers but object oriented languages use access modifiers like private, public, and protected.

Interviewer: What is encapsulation? **Candidate:** Encapsulation means binding data together in one single unit. Let's have a real life example: the word is capsule. So capsule is wrapping data into a single unit and providing the protection by using plastic cover. Same thing happens with the programming. Class is wrapping data and member functions/methods in single unit and providing the protection like by access modifiers public, private, and protected.

Interviewer: What is tightly encapsulation in Java? **Candidate:** Means if all data from class are private declared, then it is to be a tightly encapsulated class.

Interviewer: How can we access private members of the class? **Candidate:** This is actually possible by using the getter and setter methods. You can access the private data.

Interviewer: Why we should go for inheritance? **Candidate:** If we want to extend the properties of one class in another class, it's called inheritance. For example, we having class A and class A having data like addition and subtraction and having a method. We can inherit those into child classes using "extends" keyword.

Interviewer: How many types of inheritance in Java? **Candidate:** There are five types of inheritance in programming: single inheritance, multi-level, then hierarchical inheritance, multiple inheritance, and hybrid. Java support only three: single, multi-level and hierarchical. Multiple and hybrid inheritance is not supported by Java.

Interviewer: Why multiple inheritance is not supported by Java? **Candidate:** Because multiple inheritance creates ambiguity. Let's say for example we have two classes and both classes having same method and if we inherit those in a child class, so child class will get confused which class method should I call.

Interviewer: What is polymorphism? **Candidate:** Poly means many and morph means forms. Having multi-forms is nothing but polymorphism. Like for example in the real world example, a person. A person in class behave like student, a person in market behave like a customer, a person in bus behave like a passenger. Person is the same but having different behavior. In Java we have two types: compile-time polymorphism and runtime polymorphism. Method overriding is the example of runtime polymorphism and overloading is the example of compile-time polymorphism.

Interviewer: Explain static keyword in Java. **Candidate:** Static is used for the memory management. If we are using it with a variable, it is going to create only one copy of that variable. If a method is static, then it will be loaded at class loading and we have to access that method by using the class name. We also have static blocks which execute before the main method. Non-static blocks execute when we create an object.

Interviewer: What is a constructor? **Candidate:** It is used to initialize the object and it has same name as class and no return type. If you don't write any constructor, compiler creates a default constructor with no parameters. There are two types: default constructor (no arguments) and parameterized constructor (at least one argument).

Interviewer: How do you allocate memory to an object? **Candidate:** By using the "new" keyword.

Interviewer: What is the difference between a constructor and a method? **Candidate:** The main difference is a constructor is called automatically and the method you have to call explicitly.

Interviewer: What is the use of "bin" and "lib" in JDK? **Candidate:** "bin" contains all the tools like javac, and "lib" contains packages and APIs.

Interviewer: How many ways to pass arguments? **Candidate:** There are two ways: call by value and call by reference.

Interviewer: Explain access modifiers in Java. **Candidate:** Public is accessible to all classes and packages. Private is not possible to access outside class. Protected will be accessed outside class and in sub-packages. Default is accessible for the packages only.

Interviewer: What is garbage collection and how can we call it explicitly? **Candidate:** It is for the deallocating system memory. When an object is unreferenced, it is eligible for garbage collection. We can call it in Java by using the `System.gc()` method.

Interviewer: What is the use of "this" keyword? **Candidate:** It indicates current working object. If we are having a constructor and variable name is same, then to assign the value to class variables we use "this". We can also call another constructor in same class using "this".

Interviewer: What is the "super" keyword in Java? **Candidate:** It is used with variable, method, and constructor to refer to parent class properties in child class.

Interviewer: What are Var-args methods? **Candidate:** Var-args are used for passing any number of parameters. You can pass zero or many arguments.

Interviewer: What is an inner class? **Candidate:** A class inside another class is called an inner class. There is also an anonymous inner class which is a class having no name.

Interviewer: What is a wrapper class? **Candidate:** If you want to represent primitive data type into object type, we have to use wrapper classes like Integer for int, Character for char, and Double for double. They are used in the collection framework.

Interviewer: What is a factory method? **Candidate:** A factory method is a method that returns the current class object or different class object.

Interviewer: Explain the concept of Package. **Candidate:** A package is a collection of classes and interfaces. It is used to organize classes, like physical folders.

Interviewer: What is reflection? **Candidate:** It is used to get information about classes at runtime, like the name of the class, names of methods, and names of variables.

Interviewer: What are the types of imports? **Candidate:** Normal import and static import. In normal import like `java.util.*` all classes are available. In static import we can import static members specifically.

Interviewer: What is an Interface in Java? **Candidate:** It is used to achieve 100% abstraction and multiple inheritance. For example, if we have an interface Bank and

classes like SBI and Union Bank implementing it, it's compulsory for them to follow the interface rules. It provides multiple solutions for the same problem.

Interviewer: Can we write logic inside an interface in Java 8? **Candidate:** Yes, in Java 8 we can write static and default methods with logic inside the interface.

Interviewer: What is the difference between an abstract class and an interface?

Candidate: Inside abstract class we have partial implementation (abstract and non-abstract methods), but inside interface we provide 100% abstraction (before Java 8). Abstract classes support multi-level inheritance but not multiple inheritance, while interfaces support multiple inheritance.

Interviewer: What is a marker interface? **Candidate:** An interface having no body/methods is a marker interface. Examples are Serializable and Cloneable. It is for providing special features by the JVM.

Interviewer: Can you have a class inside a method? **Candidate:** Yes, we can have local inner classes inside a method.

String handling and Multithreading

Interviewer: How many classes are there in Java by which we can perform a string manipulation? **Candidate:** There are four classes for the string manipulation in Java. First one is String, second is StringBuffer, then third is StringBuilder, and fourth is StringTokenizer.

Interviewer: What is string handling in Java? **Candidate:** Basically, characters enclosed in double quotes is called a string, and to perform different operations for it is nothing but string handling. For example, joining two strings, or calculating length, or finding specific character—such operations where there is the string only.

Interviewer: What is the difference between String and StringBuffer? **Candidate:** String is a constant and immutable specific character. It is not modified in the string constant pool, but in StringBuffer, we can perform modification in the same memory. String is a constant class, and StringBuffer is a class where we can perform modification in the same memory.

Interviewer: How many ways are there to create a string in Java? **Candidate:** There are two ways we can create the string in Java. One is by using new operator and second is without using new operator. For example, using new operator: `String s1 = new String("Java Programming")`. Without using new operator (String literal): `String str = "Java"`.

Interviewer: What is the difference between immutability and mutability? **Candidate:** String is an immutable class, which means once you create an object of this string class, we cannot perform modification in the same memory. But in mutability, we can perform modification in the same memory.

Interviewer: When should we go for String, StringBuffer, and StringBuilder? **Candidate:** If we don't want to perform modification in the same memory, we should go for String. But if we want to perform activity in the same memory, then we will go for StringBuffer and StringBuilder. The difference between StringBuffer and StringBuilder is synchronization. StringBuffer is synchronized (single-threaded), and StringBuilder is non-synchronized (multi-threaded).

Interviewer: Why is String immutable in Java? **Candidate:** It is immutable for several reasons. First one is security. Security means we have numbers of parameters like username, password, URL, and database connections. If the user is using the string, it should not be changed. For example, if we are storing the password in the application, and the user changes it, it could be a security risk. Also, class loading: if it is related to a custom class, it will affect the security. Immutability increases security in hashing as well.

Interviewer: How can we convert String to StringBuffer and vice versa? **Candidate:** We can convert String to StringBuffer using the constructor. We have to pass the string object. And to convert StringBuffer to String, we can use the toString() method.

Interviewer: Why do we use a character array for storing a password instead of a String?

Candidate: String is an immutable class and is stored in the string constant pool. It will be available for a long time in the application. Anyone can access that memory. But in the case of a character array, once used, we can set it to blank or empty, and that particular data will not be visible in the application. So for security, we use character arrays.

Interviewer: Why is String immutable and final in Java? **Candidate:** It is an immutable and final class because if a class is declared final, we cannot create its subclass. This increases security. For example, for sensitive information like database password and username, String immutability is used in multi-threading without any synchronization.

Interviewer: What is the String Constant Pool (SCP)? **Candidate:** SCP is a separate area inside the heap memory where string literals are stored. If a string is already available in the pool, it will not create another memory, it will reuse the reference. If not available, it will create another one.

Interviewer: How can we get a character array from a String? **Candidate:** By using the toCharArray() method.

Interviewer: How many objects will be created for String s1 = "Ashish"; String s2 = "Ashish";? **Candidate:** Only one object will be created in the String Constant Pool.

Interviewer: Where is the String Constant Pool exactly located in memory? **Candidate:** Inside the heap memory.

Interviewer: What is the difference between Java Strings and C++ Strings? **Candidate:** In C++, the string is ended with a null character (\0), but in Java, it is not ended with a null character. In C++, we have to use character arrays, but in Java, we have the String class.

Interviewer: How many objects are created using the new keyword? **Candidate:** Two objects are created: one in heap memory and another one in the String Constant Pool.

Interviewer: What is Multi-programming? **Candidate:** Multi-programming is the process of execution of programs simultaneously.

Interviewer: What is a thread and what is Multi-threading in Java? **Candidate:** A thread is a sequential execution and it is a light-weight process. It does not occupy the same memory area. Multithreading is the execution of multiple threads concurrently. For example, in MS Word, typing, spell-checking, and saving are all different threads working simultaneously.

Interviewer: What are the applications of Multi-threading? **Candidate:** Video games, animations, multimedia graphics, and client-server applications.

Interviewer: How can we create a thread in Java? **Candidate:** There are two ways: by extending the Thread class and by implementing the Runnable interface.

Interviewer: What is the difference between start() and run() methods? **Candidate:** When calling start(), it will execute the Thread class's start logic, create a new thread, and internally call the run() method. If you call run() directly, no new thread is created; it behaves like a normal method call.

Interviewer: What is the Thread Scheduler? **Candidate:** It manages the execution of multiple threads on a single CPU. It decides which thread gets the chance to execute based on algorithms like First-Come-First-Serve, Time-Slicing, or Round-Robin. We cannot expect a fixed execution order; it depends on the scheduler and operating system.

Interviewer: Explain the lifecycle states of a thread. **Candidate:** New (after creating thread object), Runnable (after calling start()), Running (when thread scheduler calls run()), Waiting/Blocked/Sleeping (when wait(), sleep(), or suspend() is called), and Dead (after execution or calling stop()).

Interviewer: How can we set and get the name of a thread? **Candidate:** Every thread has a name generated by the JVM. We can set a custom name using the setName() method and retrieve it using the getName() method.

Interviewer: Explain Thread priority. **Candidate:** Java threads have priorities ranging from 1 (MIN_PRIORITY) to 10 (MAX_PRIORITY), with a default of 5 (NORM_PRIORITY). A higher priority thread gets a chance to execute first. Example: A traffic policeman on a road. If a minister's car arrives (high priority), they get the chance to pass first while others wait.

Interviewer: How to prevent a thread from execution? **Candidate:** By using yield(), join(), and sleep() methods.

Interviewer: Explain the yield() method with an example. **Candidate:** yield() causes the current thread to pause and give a chance to other waiting threads of the same or higher priority. Example: If I am using my mobile for a call, and a message arrives, the call is the current priority. But if I decide to pause the call to let the message be delivered, that is like yielding.

Interviewer: Explain the join() method with an example. **Candidate:** If a thread wants to wait until another thread completes its execution, it uses join(). Example: If Thread 1 (T1) calls T2.join(), T1 will enter the waiting state until T2 finishes its work.

Interviewer: What is the sleep() method? **Candidate:** If a thread doesn't want to perform any operation for a particular amount of time, it uses sleep(). It requires handling InterruptedException with a try-catch block.

Interviewer: How can a thread interrupt another thread? **Candidate:** A thread can interrupt a sleeping or waiting thread using the interrupt() method. Example: Four friends are waiting for a bus and decide to sleep for 30 minutes. If one friend wakes up because the bus arrives, they interrupt the others to wake them up.

Interviewer: What is synchronization? **Candidate:** It is the process of allowing only one thread to access a shared resource at a time to prevent data inconsistency. Example: Railway ticket reservation system. If only 4 tickets are available and two people request 3 and 2 tickets simultaneously, synchronization ensures only one request is processed at a time using locks.

Interviewer: What is a Shutdown Hook? **Candidate:** It is used to perform clean-up activities (like closing databases or releasing resources) before the JVM shuts down. We use addShutdownHook() and removeShutdownHook().

Interviewer: What is a Daemon Thread? **Candidate:** A thread that executes in the background to provide support to foreground threads. Examples: Garbage Collector and Exception Handler. They are typically low priority and stop automatically when all user threads finish.

Interviewer: Is it possible to start the same thread twice? **Candidate:** No, it will throw an IllegalStateException.

Interviewer: Can we override the start() method? **Candidate:** Yes, but if you do, the JVM will execute your overridden start() method, and a new thread will not be created.

Interviewer: What is the difference between notify() and notifyAll()? **Candidate:** notify() wakes up a single thread waiting on the object's monitor, while notifyAll() wakes up all threads waiting on that monitor.

Interviewer: How do threads communicate with each other? **Candidate:** Threads communicate using wait(), notify(), and notifyAll() methods to share resources and status.

Interviewer: What is Deadlock? **Candidate:** A situation where two threads are waiting for each other to release resources. Example: T1 has a lock on Object 1 and waits for Object 2; T2 has a lock on Object 2 and waits for Object 1. Both wait forever.

Interviewer: What is the difference between Runnable and Callable? **Candidate:** Runnable (introduced in Java 1.0) has a run() method that does not return a value or throw checked exceptions. Callable (introduced in Java 1.5) has a call() method that returns a result and can throw checked exceptions.

Interviewer: What is a Semaphore? **Candidate:** It is used to limit the number of concurrent threads accessing a resource. If permits are available, a thread can proceed; otherwise, it must wait.